VibeMeGood — Player Thumbnail Photos

Add-on spec. Low complexity, high visual impact.

WHY IT'S EASY

The PrizePicks API already returns `image_url` for every player in the sync response. The `syncPpLines()` function parses `pAttr` (player attributes) but discards this field. All that is needed is: store it, pass it through, display it with a fallback.

No new API. No new credentials. No new cron job.

STEP 1: Database — Add imageUrl to players table

File: `lib/db/src/schema/players.ts`

```
// Add one field:
export const playersTable = pgTable("players", {
  id:          serial("id").primaryKey(),
  sport:       text("sport").notNull(),
  fullName:    text("full_name").notNull(),
  firstName:   text("first_name").notNull(),
  lastName:    text("last_name").notNull(),
  teamId:      integer("team_id"),
  position:    text("position"),
  status:      text("status").notNull().default("active"),
  imageUrl:    text("image_url"),              // ← ADD THIS
  externalIds: jsonb("external_ids"),
  createdAt:   timestamp("created_at").defaultNow().notNull(),
  updatedAt:   timestamp("updated_at").defaultNow().notNull(),
});
```

Run the migration after adding the column.

STEP 2: PP Sync — Capture image_url

File: `artifacts/api-server/src/lib/sync/prizepicks.ts`

```
// The pAttr object already contains image_url — just capture it:
const imageUrl = (pAttr.image_url as string) || null;

// When upserting a new player, include it:
[player] = await db.insert(playersTable).values({
  sport, fullName: playerName,
  firstName: parts[0] || "",
  lastName: parts.slice(1).join(" ") || "",
  teamId, status: "active",
  imageUrl,                                    // ← ADD
  externalIds: { pp_id: proj.relationships?.new_player?.data?.id },
}).returning();

// When a player already exists, update imageUrl if it changed:
} else if (
  (teamId && player.teamId !== teamId) ||
  (imageUrl && player.imageUrl !== imageUrl)   // ← ADD this condition
) {
  await db.update(playersTable)
    .set({ teamId, imageUrl: imageUrl ?? player.imageUrl, updatedAt: new Date() }
    .where(eq(playersTable.id, player.id));
}
```

Fallback for sports where PP doesn't provide photos:

NBA Stats API has official headshots free of charge. Use as a backup when PP doesn't supply an image. NBA player IDs are already being stored in `player_id_mapping` .

```
// In syncPpLines(), after upserting player with no imageUrl:
if (!imageUrl && player.sport === "NBA") {
  // Pull NBA ID from player_id_mapping if it exists
  const [mapping] = await db.select().from(playerIdMappingTable)
    .where(and(eq(playerIdMappingTable.playerId, player.id), eq(playerIdMappingTa
    .limit(1);
  if (mapping?.externalId) {
    const nbaHeadshot = `https://cdn.nba.com/headshots/nba/latest/260x190/${mappi
    await db.update(playersTable)
      .set({ imageUrl: nbaHeadshot })
      .where(eq(playersTable.id, player.id));
  }
}
```

STEP 3: Pass imageUrl through market-intel and slate routes

File: `artifacts/api-server/src/routes/market-intel.ts`

```
// The player join already returns the full player object.
// Add imageUrl to the return object:
return {
  ppLineId: row.line.id,
  playerName: row.player.fullName,
  imageUrl: row.player.imageUrl ?? null,       // ← ADD
  // ... rest of fields
};
```

File: `artifacts/api-server/src/routes/slate.ts`

```
// In the rows.map() return object:
return {
  ppLineId: line.id,
  playerName: player?.fullName ?? "Unknown",
  imageUrl: player?.imageUrl ?? null,           // ← ADD
  teamAbbr: playerTeam?.abbreviation ?? null,
  // ... rest of fields
};
```

STEP 4: Add imageUrl to EntryPick type

File: `artifacts/prizepicks/src/lib/entry-context.tsx`

```
export interface EntryPick {
  ppLineId:       number;
  playerId:       number;
  playerName:     string;
  imageUrl:       string | null;               // ← ADD
  teamAbbr:       string | null;
  statType:       string;
  lineValue:      number;
  lineType:       string;
  direction:      "more" | "less";
  yourProjection: number | null;
  pOver:          number | null;
  edgeScore:      number | null;
  actionTag:      string | null;
```

```
}
```

When adding a pick to entry (in Slate Board), include `imageUrl` in the pick object passed to `addPick()`.

STEP 5: PlayerAvatar component

File: `artifacts/prizepicks/src/components/ui/player-avatar.tsx`

```
import { useState } from "react";

// Generates a consistent color from the player's name
function avatarColor(name: string): string {
  const palette = [
    "bg-blue-800",    "bg-violet-800", "bg-emerald-800",
    "bg-rose-800",    "bg-amber-800",  "bg-cyan-800",
    "bg-indigo-800",  "bg-teal-800",   "bg-fuchsia-800",
  ];
  const hash = name.split("").reduce((acc, c) => acc + c.charCodeAt(0), 0);
  return palette[hash % palette.length];
}

function initials(name: string): string {
  const parts = name.trim().split(/\s+/);
  if (parts.length === 1) return parts[0][0]?.toUpperCase() ?? "?";
  return `${parts[0][0]}${parts[parts.length - 1][0]}`.toUpperCase();
}

interface PlayerAvatarProps {
  name:     string;
  imageUrl: string | null | undefined;
  size?:    "xs" | "sm" | "md" | "lg";
  className?: string;
}

const SIZE_MAP = {
  xs: { container: "w-6 h-6",   text: "text-[8px]",  ring: "ring-1" },
  sm: { container: "w-8 h-8",   text: "text-[10px]", ring: "ring-1" },
  md: { container: "w-10 h-10", text: "text-xs",      ring: "ring-2" },
  lg: { container: "w-16 h-16", text: "text-base",    ring: "ring-2" },
};

export function PlayerAvatar({ name, imageUrl, size = "sm", className = "" }: Pla
  const [imgError, setImgError] = useState(false);
```

```
  const { container, text, ring } = SIZE_MAP[size];
  const showImage = imageUrl && !imgError;

  return (
    <div
      className={`
        ${container} rounded-full overflow-hidden shrink-0
        ${ring} ring-slate-700
        ${showImage ? "" : avatarColor(name)}
        flex items-center justify-center
        ${className}
      `}
    >
      {showImage ? (
        <img
          src={imageUrl}
          alt={name}
          className="w-full h-full object-cover"
          onError={() => setImgError(true)}
          loading="lazy"
          decoding="async"
        />
      ) : (
        <span className={`${text} font-bold text-white/90 font-mono select-none`}>
          {initials(name)}
        </span>
      )}
    </div>
  );
}
```

STEP 6: Drop it in everywhere

Slate Board rows

File: `artifacts/prizepicks/src/pages/slate-board.tsx`

```
// Import the component at the top:
import { PlayerAvatar } from "@/components/ui/player-avatar";

// In the TableCell that shows the player name (currently just text):
<TableCell>
  <div className="flex items-center gap-2.5">
    <PlayerAvatar name={row.playerName} imageUrl={row.imageUrl} size="sm" />
```

```
    <div>
      <div className="font-bold text-sm">{row.playerName}</div>
      {/* DQ badge stays here */}
    </div>
  </div>
</TableCell>
```

Entry Builder picks list

File: `artifacts/prizepicks/src/pages/entry-builder.tsx`

```
// In the picks list div (currently: w-5 h-5 rounded-full bg-slate-800 with pick
// Replace the number circle with PlayerAvatar:
<PlayerAvatar name={pick.playerName} imageUrl={pick.imageUrl} size="sm" />

// The pick number is redundant once you have the photo — remove it or put the
// number as a tiny overlay badge if you want both.
```

Prop Detail Sheet header

File: `artifacts/prizepicks/src/components/prop-detail-sheet.tsx`

```
// In the SheetHeader section where the player name and stat are shown:
<div className="flex items-center gap-3">
  <PlayerAvatar name={detail.player.fullName} imageUrl={detail.player.imageUrl} s
  <div>
    <SheetTitle className="text-lg font-bold">{detail.player.fullName}</SheetTitl
    <SheetDescription className="font-mono text-xs">
      {detail.ppLine.statType} · {detail.ppLine.lineValue}
    </SheetDescription>
  </div>
</div>
```

Streak Tracker

File: `artifacts/prizepicks/src/pages/streaks.tsx`

```
// In the streak card/row that shows playerName:
<div className="flex items-center gap-2.5">
  <PlayerAvatar name={streak.playerName} imageUrl={streak.imageUrl} size="sm" />
  <div>
    <div className="font-semibold text-sm">{streak.playerName}</div>
    <div className="text-xs text-muted-foreground font-mono">{streak.teamAbbr} ·
  </div>
```

```
</div>
```

For streaks to have `imageUrl`, add it to the streaks route response in `artifacts/api-server/src/routes/streaks.ts` — join the players table and return `player.imageUrl`.

Dashboard top picks

File: `artifacts/prizepicks/src/pages/dashboard.tsx`

In any card that lists props by player name, add a `PlayerAvatar size="sm"` alongside the name.

WHAT NOT TO DO

- Do not load full-resolution images in table rows. The PP-provided `image_url` and NBA CDN URLs are already appropriately sized (260×190 for NBA). Let the browser resize via the `w-8 h-8` container.
- Do not block rendering waiting for images. The `loading="lazy"` attribute and the initials fallback on error handle this.
- Do not try to fetch images for every player on page load. Images load passively as rows scroll into view via lazy loading.
- Do not hardcode sport-specific image APIs. The PP image URL works for all sports PP supports. NBA CDN is a supplemental fallback only.

ACCEPTANCE TEST

1. Slate Board shows a circular player thumbnail (or colored initials) next to every player name.
2. A player with no PP image and no NBA mapping shows their initials in a colored circle. The circle color is consistent — it does not change between page loads.
3. Prop Detail Sheet shows a larger (64px) thumbnail in the header next to the player name.
4. Entry Builder picks list shows small thumbnails instead of (or alongside) the numbered circles.
5. An image URL that 404s or fails to load gracefully falls back to the initials — no broken

image icon, no layout shift.

6. Running Force Sync updates `imageUrl` in the players table for any player whose PP image changed.

COMPLEXITY RATING

Low. One schema column, two sync lines, one reusable component, six drop-in placements. The component handles all edge cases (error, missing URL, long names) in one place. Everything downstream just passes `name` and `imageUrl` — no other changes needed.